

A Shared Conversation Governor

for OpenClaw, OmegaClaw, and OmegaHive

Reducing token use and conversational noise without reducing useful capability

Revised design with conservative activation, topology fallbacks, and semantic safeguards

Architecture, detailed design, rollout strategy, and implementation plan

Revision 2: shadow-first deployment and control-plane safety refinements

Prepared for Ben Goertzel

July 24, 2026

Scope. This document proposes a shared communication control plane for the current two-agent proto-hive, consisting of an OpenClaw agent with extensive skills (Protocosmobot) and an OmegaClaw agent that uses the OpenClaw gateway. Both agents currently use smart-ish model routers. The design extends this setup with event admission, responder arbitration, deduplication, private bot-to-bot review, update aggregation, and output governance. The preferred shared pre-inference placement is treated as an empirical deployment question rather than an assumption; distributed and weak-fallback topologies are specified. The revision also adds governor-cost budgets, thread-level ownership hysteresis, an explicit semantic firewall for egress processing, predeclared activation gates, and a human override grammar. It describes how the architecture can scale to a richer OmegaHive spanning many agents and both AI-only and mixed human/AI channels.

Contents

1	Executive Summary	1
2	Motivation and Empirical Problem	2
2.1	The key distinction: verbosity versus message creation	2
2.2	Observed failure classes in the current proto-hive	3
2.3	Why prompt-only fixes are insufficient	3
3	Goals, Non-Goals, and Design Principles	4
3.1	Primary goals	4
3.2	Non-goals	4
3.3	Core design principles	5
4	Deployment Topology as an Empirical Discovery Result	5
4.1	Topology A: centralized pre-inference governor	5
4.2	Topology B: coordinated local governors	6
4.3	Topology C: weak ingress/egress fallback	6
4.4	Placement invariants	6
5	High-Level Architecture	7
5.1	Proposed topology	7
5.2	Logical architecture versus physical placement	8
5.3	Control plane versus cognition plane	8
6	Detailed Component Design	8
6.1	Event normalization and provenance envelope	8
6.2	Event admission	9
6.2.1	Deterministic drop conditions	9
6.2.2	Admission decision object	10
6.3	Addressing, ownership, and response leases	10
6.3.1	Ownership precedence and conversational hysteresis	10
6.3.2	Lease semantics	11
6.4	Task and thread lifecycle	11
6.5	Idempotency and duplicate detection	12
6.5.1	Exact idempotency	12
6.5.2	Near-duplicate and novelty detection	12
6.6	Extended smart router	13
6.6.1	Governor cost cascade and budget	13
6.7	Private bot-to-bot review bus	14
6.8	Context construction and project-state snapshots	14
6.9	Cron workers and update aggregation	15
6.10	Incident manager	15
6.11	Probe-before-prose gate	16
6.12	Egress governor	16
6.12.1	Acknowledgment-like output and structured no-send	16
6.12.2	Semantic firewall for egress processing	17
6.12.3	Candidate outbound schema	17
7	Agent-Specific Integration	18

7.1	Protocosmobot as execution and evidence agent	18
7.2	OmegaClaw as synthesis and reasoning agent	19
7.3	When both agents should contribute	19
8	Algorithms and Decision Procedures	20
8.1	Inbound event processing	20
8.2	Public owner resolution	21
8.3	Egress processing	21
8.4	Daily digest construction	22
9	Response Classes and Progressive Disclosure	23
10	Channel Policies	23
11	Human Override Grammar	24
12	Failure Modes and Safety Considerations	25
12.1	False silence	25
12.2	Over-centralization	25
12.3	Lease deadlock or abandonment	25
12.4	Semantic deduplication errors	25
12.5	Private review leakage	25
12.6	Human confusion about who answered	25
12.7	Governor overhead and negative savings	26
12.8	Ownership thrashing	26
12.9	Semantic mutation by the governor	26
13	Observability and Evaluation	26
13.1	Core metrics	26
13.2	Predeclared activation and go/no-go gates	27
13.3	Audit records	27
14	Deployment and Migration Strategy	28
14.1	Phase 0: baseline instrumentation only	28
14.2	Phase 1: shadow decisions and false-silence audit	28
14.3	Activation Gate 1: explicit approval	28
14.4	Phase 2: deterministic suppression	28
14.5	Phase 3: shared response leases with hysteresis	29
14.6	Phase 4: private review bus	29
14.7	Phase 5: daily digest and compact project state	29
14.8	Phase 6: calibrated semantic novelty and adaptive budgets	29
15	Extension to a Richer OmegaHive	29
15.1	From a two-agent governor to a communication fabric	29
15.2	Typed channels and boundary brokers	30
15.3	Topic and role subscriptions	30
15.4	Bounded deliberation committees	30
15.5	Attention budgets	31
15.6	Cross-channel state and duplicate prevention	31
15.7	Reputation and confidence without chatter	31
15.8	Governance, moderation, and social behavior	31

16 Recommended Initial Configuration for the Current Proto-Hive	32
17 Conclusion	33
A Detailed Implementation Plan for an OpenClaw Coding Agent	34
A.1 Implementation mandate	34
A.2 Operating constraints for the implementing agent	34
A.3 Phase A: repository and runtime discovery	34
A.3.1 Tasks	34
A.3.2 Deliverable	35
A.3.3 Acceptance criteria	35
A.4 Phase B: event schema and durable ledger	36
A.4.1 Tasks	36
A.4.2 Suggested module layout	36
A.4.3 Acceptance criteria	37
A.5 Phase C: transcript-derived replay corpus	37
A.5.1 Tasks	37
A.5.2 Example fixture	38
A.5.3 Acceptance criteria	38
A.6 Phase D: shadow admission engine	38
A.6.1 Tasks	38
A.6.2 Acceptance criteria	38
A.7 Phase E: hard no-op suppression	39
A.7.1 Initial suppression classes	39
A.7.2 Required safeguards	39
A.7.3 Acceptance criteria	39
A.8 Phase F: response leases and ownership	39
A.8.1 Tasks	39
A.8.2 Acceptance criteria	40
A.9 Phase G: human overrides and semantic egress firewall	40
A.9.1 Tasks	40
A.9.2 Acceptance criteria	40
A.10 Phase H: private review bus	41
A.10.1 Tasks	41
A.10.2 Acceptance criteria	41
A.11 Phase I: project snapshots and digest migration	41
A.11.1 Tasks	41
A.11.2 Acceptance criteria	41
A.12 Phase J: incident coalescing	42
A.12.1 Tasks	42
A.12.2 Acceptance criteria	42
A.13 Phase K: semantic deduplication and calibration	42
A.13.1 Tasks	42
A.13.2 Acceptance criteria	42
A.14 Phase L: metrics dashboard and policy tuning	42
A.15 Feature flags	43
A.16 Testing strategy	43
A.16.1 Unit tests	44
A.16.2 Property tests	44
A.16.3 Replay tests	44

A.16.4 Canary tests	44
A.17 Suggested commit sequence	44
A.18 Additional implementation gates	45
A.19 Definition of done for the first production release	45
A.20 Implementation-agent reporting format	46
B Example Policy Configuration	46
C Test Matrix	47
D Future Research Directions	48

1 Executive Summary

The current proto-hive contains two powerful but partially overlapping conversational actors:

- **Protocosmobot**, an OpenClaw agent with broad operational skills, tools, project access, cron workers, and the ability to inspect and modify external state.
- **OmegaClaw**, a more reflective or synthetic agent that uses the OpenClaw gateway and can provide deeper conceptual analysis, criticism, and long-form reasoning.
- Each agent has a smart-ish router that chooses which language model should answer a given message.

This arrangement has substantial functional value, but its present communication dynamics allow avoidable token use and noise. The major waste is not simply that useful answers are occasionally too long. A larger problem is that the system often creates messages that should never have been generated: acknowledgments of acknowledgments, reactions to status messages, narration of tool use, repeated error traces, duplicate explanations, visible `NO_REPLY` messages, and multiple public answers to one event.

The proposed solution is a shared OmegaHive Conversation Governor (OCG) placed, where the actual runtime permits, at a common boundary before model selection and after model generation. The existence of a truly shared pre-inference boundary must be established during repository and runtime discovery. If it does not exist, the same policy is implemented through per-agent local governors coordinated by a shared durable ledger, or through a weaker ingress/egress fallback that still prevents duplicate public output. The governor adds two control surfaces:

1. An **ingress and admission governor** decides whether any agent or language model should be invoked, which agent owns the public response, whether a private reviewer is useful, and what output contract applies.
2. An **egress governor** suppresses no-op output, rejects duplicates, assembles multipart responses, enforces response contracts, and prevents operational chatter from becoming public conversation.

The smart router is correspondingly expanded. Instead of choosing only among language models, it chooses among actions:

`DROP | LOG_ONLY | TEMPLATE | CHEAP_MODEL | NORMAL_MODEL | DEEP_MODEL`

The most important consequence is that many events terminate before any LLM call. A receipt, duplicate system event, repeated cron status, sibling echo, or non-addressed bot message can be recorded without invoking a model or posting a reply.

For the current two-agent system, the default responsibility split is:

- Protocosmobot is the **execution and evidence agent**. Its tools and workers emit structured facts, artifacts, blockers, approvals, and state changes.
- OmegaClaw is the **synthesis and reasoning agent**. It receives compact state plus selected evidence rather than raw operational streams.
- The governor grants exactly one **public response lease** per event or task state. A second agent may act as a private reviewer, but does not independently answer the human unless explicitly invited.

The proposed architecture preserves utility because it does not force all reasoning to become short or shallow. Deep analysis remains available when requested or justified. The design instead removes

redundant invocations, moves operational detail into durable artifacts and logs, makes silence an actual control action, and uses progressive disclosure for optional detail. It is activated conservatively: Phase 0 is instrumentation only, Phase 1 computes shadow decisions without changing delivery, and no suppression or topology-changing behavior is enabled until replay and live shadow data quantify false-silence risk, governor overhead, and projected net savings, followed by explicit human approval. The egress governor may perform formatting and other demonstrably non-semantic transformations, but it is forbidden from silently adding, deleting, or altering substantive claims.

For a larger OmegaHive, the same principles generalize into a multi-channel communication fabric with:

- typed channels and privacy labels;
- topic- and role-based subscriptions;
- local channel governors coordinated by a global event ledger;
- internal AI-only deliberation rooms;
- mixed human/AI rooms with one or more designated public spokesagents;
- bounded review committees rather than uncontrolled many-agent reply cascades;
- attention budgets, response leases, incident aggregation, and durable handoff artifacts.

The implementation can be staged safely. The first deployment should run in shadow mode, recording what would have been suppressed without actually suppressing anything. Once false-silence rates are acceptably low, deterministic no-op suppression and repeated-event coalescing can be activated, followed by shared response leases, private review requests, daily digest aggregation, and eventually multi-channel OmegaHive federation.

2 Motivation and Empirical Problem

2.1 The key distinction: verbosity versus message creation

There are two separable sources of token waste:

1. **Essay amplification:** a useful response is longer than necessary for the immediate decision.
2. **Message amplification:** a response is generated even though no response was needed, or several agents respond to the same event.

Length limits and concise prompting primarily address essay amplification. They are useful, but they do not address the larger structural failure modes. A bot that generates `NO_REPLY`, “noted,” or “nothing to add” has already consumed model tokens. If this text is then posted into a shared channel, it can provoke another agent to spend tokens interpreting and acknowledging it.

The central design principle is therefore:

The cheapest and least noisy response is not a shorter LLM response. It is a correctly suppressed LLM invocation and no outbound message.

2.2 Observed failure classes in the current proto-hive

The transcripts exhibit recurring classes of behavior that are individually understandable but collectively expensive:

Acknowledgment loops

One agent says “noted,” another acknowledges the note, and the first confirms that the acknowledgment is already covered.

Visible silence markers

A bot outputs `NO_REPLY`, “staying quiet,” or “nothing to add” into the channel instead of returning an internal no-send status.

Parallel public answers

Protocosmobot and OmegaClaw each answer the same human message, often with overlapping substance.

Tool narration

Internal steps such as “let me inspect,” “continuing provisioning,” or “now I will retry” are posted as public conversational messages.

System-event echoing

Full stack traces, fallback notices, or cron failures are repeatedly injected into a channel and repeatedly analyzed.

State reconstruction

A stateless worker rereads broad context and re-explains project history rather than consuming a compact current-state object.

Channel and identity confusion

An agent responds to a message meant for another agent, answers a question from a different project, or treats system metadata as a request.

Promise loops

A bot repeatedly announces that it is reading or preparing a result but does not deliver a concrete artifact.

Speculative prose before cheap verification

Several paragraphs are produced about an operational uncertainty that could have been resolved by a simple file, process, commit, or job-status check.

Over-reporting incremental progress

Small test-count increases, routine commits, or “no material change” updates are sent many times per day.

These behaviors do not imply poor underlying reasoning. They arise because the current architecture gives each agent an incomplete local view and asks each model invocation to decide both *whether* to respond and *what* to say. The result is a conversational tragedy of the commons: locally reasonable agents jointly produce global noise.

2.3 Why prompt-only fixes are insufficient

Both agents should receive stronger communication instructions, but prompts alone cannot reliably enforce system-wide invariants:

- An agent cannot know that a sibling already claimed a message unless there is shared state.
- A model cannot save the tokens used to decide that it should say `NO_REPLY`; only pre-inference admission can do that.

- Retry and reconnect events may cause the same input to be processed more than once unless idempotency is enforced outside the model.
- A model may correctly identify an echo loop and still produce another message explaining the echo loop.
- Independent routers may both choose a sensible model for the same event; they need a shared arbitration layer, not merely better model selection.

The proposed governor therefore treats communication discipline as a first-class control-plane problem.

3 Goals, Non-Goals, and Design Principles

3.1 Primary goals

The system should:

1. Reduce total model input and output tokens without materially reducing useful functionality.
2. Prevent duplicate or no-op public messages before they are generated.
3. Preserve the value of having both an operational agent and a reflective reasoning agent.
4. Ensure one clear public answer by default, even when multiple agents contribute internally.
5. Aggregate routine project progress into compact, meaningful updates.
6. Keep detailed evidence and reasoning in durable artifacts rather than repeatedly regenerating it in chat.
7. Improve traceability: every public response should have a clear triggering event, owner, state version, and rationale.
8. Scale from the present two-agent proto-hive to a larger OmegaHive with many channels and agents.

3.2 Non-goals

The design is not intended to:

- force all answers into a uniformly short style;
- prevent rich philosophical, scientific, or technical dialogue when that dialogue is the point;
- replace agent-specific personalities, skills, or model routers;
- centralize all cognition in one monolithic controller;
- hide consequential disagreement among agents;
- prevent humans from explicitly requesting multiple independent answers;
- make the system silent merely to optimize cost.

3.3 Core design principles

1. **Admission before inference.** Decide whether an LLM call is needed before selecting a model.
2. **One public owner, many private contributors.** Multiple agents may assist, but one agent normally holds the public response lease.
3. **Silence is a real action.** No-send is structured control output, never user-visible prose.
4. **Delta over narrative.** Communicate changed state, decisions, blockers, and evidence; do not reconstruct unchanged history.
5. **Probe before prose.** Resolve cheap operational uncertainties before writing extensive analysis.
6. **Artifacts carry depth.** Chat carries summaries, decisions, and pointers; durable files carry long reasoning and evidence.
7. **Stateful infrastructure, stateless models.** Models may remain replaceable and session-local while the communication fabric maintains durable event, task, and project state.
8. **Fail open for usefulness only where safe.** Early deployment should prefer logging and shadow decisions over aggressive suppression; suppression becomes stricter only after measurement.
9. **Human intent is authoritative.** Explicit user addressing, request for multiple viewpoints, or request for live updates overrides default quietness policies.

4 Deployment Topology as an Empirical Discovery Result

The current system appears approximately as follows:

```
Telegram and other channels
|
+--> Protocosmobot / OpenClaw
|   - skills and tools
|   - project workers and cron jobs
|   - smart-ish model router
|
+--> OmegaClaw
    - reflective/synthetic reasoning
    - uses an OpenClaw gateway
    - smart-ish model router
```

The architecture must *not* assume, without verification, that both agents traverse one inspectable pre-inference boundary. Whether OmegaClaw inbound events pass through the OpenClaw gateway before prompt construction and model invocation is a repository and runtime discovery question. The implementation must select among three supported placements.

4.1 Topology A: centralized pre-inference governor

This is the preferred arrangement:

```
all inbound events
|
v
```

```

shared governor and ledger
|
+--> Protocosmobot router and agent
+--> OmegaClaw router and agent

```

It provides the strongest savings because ownership, idempotency, no-send decisions, and context selection occur before either agent invokes an LLM.

4.2 Topology B: coordinated local governors

If the agents have distinct inbound paths but can access one durable coordination service, each agent receives a local pre-inference governor:

```

Protocosmobot events --> local governor A --+
                                     +--> shared ledger and lease service
OmegaClaw events      --> local governor B --+

```

The local governors use atomic leases, handled-event records, thread state, and recent fingerprints from the shared ledger. This preserves one-public-owner semantics and avoids most duplicate inference, although each local boundary must be instrumented and tested independently.

4.3 Topology C: weak ingress/egress fallback

If no reliable shared or coordinated pre-inference insertion point exists, implement the minimum viable control fabric at channel ingress and shared egress:

- channel adapters filter own-message loopback and non-addressed sibling traffic;
- each agent applies local admission and response contracts;
- shared egress enforces one public response, duplicate rejection, incident coalescing, and structured no-send;
- some duplicate internal model calls may remain, but duplicate public output and recursive channel amplification are still prevented.

Topology C is a valid incremental fallback, not the target end state. Phase A must document which topology is actually feasible, why, and what cost or coverage is lost relative to Topology A.

4.4 Placement invariants

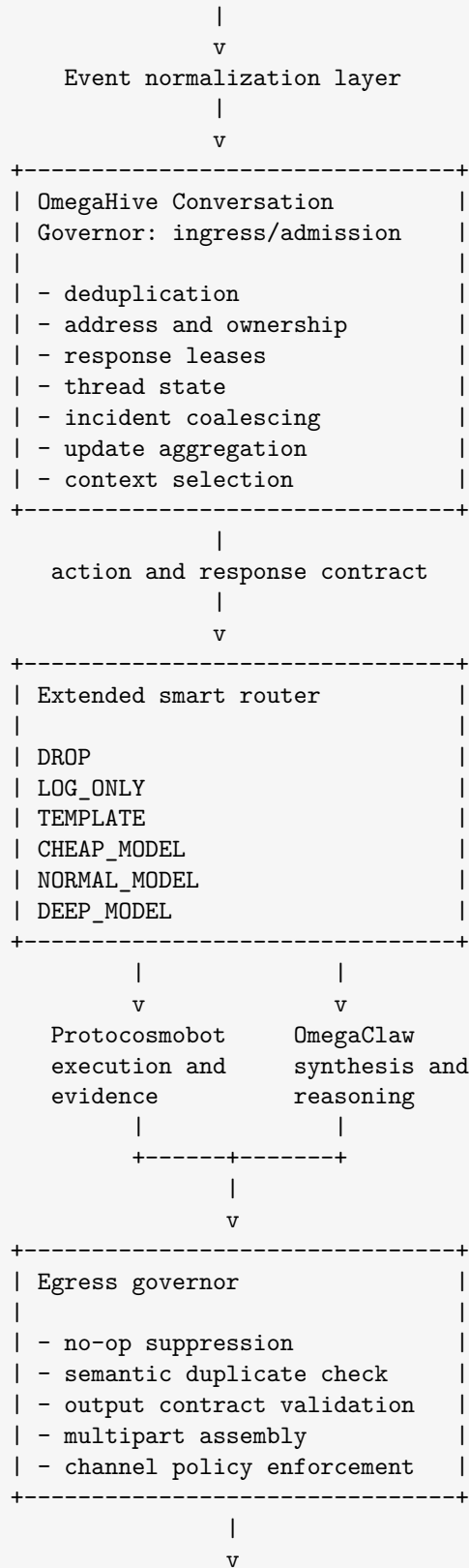
Regardless of topology:

1. suppression decisions must be auditable and reversible;
2. high-severity, explicit human, authority, cost, and safety events retain a bypass path;
3. the governor must not become a hidden substantive reasoner;
4. origin chains and event IDs must survive cross-agent and cross-channel transport;
5. the system must support shadow observation before active behavior changes.

5 High-Level Architecture

5.1 Proposed topology

External channels, cron jobs, tools, project workers



Human-visible channels

5.2 Logical architecture versus physical placement

The diagram above describes logical responsibilities, not a guaranteed single-process deployment. In Topology A, all boxes may be implemented in one shared gateway service. In Topology B, ingress admission and response-contract checks are replicated locally while lease, event, thread, and incident state remain shared. In Topology C, the shared egress layer is authoritative for public sends while local admission reduces avoidable calls where possible.

Policy semantics should therefore be factored from transport adapters. The same admission decision object, lease protocol, audit schema, and channel policy should operate across all three placements. This prevents repository discovery from forcing a redesign of the policy model.

5.3 Control plane versus cognition plane

The governor belongs to the **control plane**. It does not need to solve the user task. Its job is to decide:

- whether the event should cause any inference;
- which agent is responsible;
- whether a private review is warranted;
- how much context is needed;
- what cost and length budget applies;
- whether the output may be sent.

Protocosmobot and OmegaClaw remain in the **cognition and action plane**. They perform tools, reason, synthesize, and create artifacts. This separation is important because communication admission should be deterministic where possible, cheap, auditable, and shared across agents.

6 Detailed Component Design

6.1 Event normalization and provenance envelope

Every inbound item should be converted into a canonical event envelope. Sources include human messages, bot messages, tool results, cron completions, system errors, file arrivals, project-state changes, and internal review requests.

A proposed schema follows.

```
{
  "event_id": "evt_01J...",
  "event_type": "human_message",
  "timestamp": "2026-07-24T18:42:00Z",
  "source": {
    "channel_id": "telegram:protobots",
    "channel_type": "mixed_human_ai",
    "sender_id": "human:ben",
    "sender_kind": "human",
    "message_id": "tg:299803"
```

```

},
"addressing": {
  "explicit_agents": ["protocosmo"],
  "reply_to_agent": null,
  "reply_to_event_id": "evt_01H..."
},
"causality": {
  "caused_by_event_id": null,
  "origin_chain": ["human:ben"],
  "retry_of_event_id": null
},
"scope": {
  "project_id": "omegasim",
  "task_id": "a6-resweep",
  "thread_id": "thread:omegasim:a6",
  "state_version": 42
},
"content": {
  "text": "...",
  "attachments": [],
  "content_hash": "sha256:...",
  "semantic_fingerprint": "emb:v1:..."
},
"policy": {
  "privacy": "mixed-room",
  "urgency": "normal",
  "human_response_expected": true
}
}

```

The envelope is critical for preventing loops. When a bot's own outbound message returns through Telegram, its origin chain remains visible. A sibling agent should not treat it as a fresh human request unless it is explicitly addressed or assigned for review.

6.2 Event admission

Admission is the first decision point. It should be implemented as a sequence of cheap checks, with an optional small classifier only after deterministic checks fail.

6.2.1 Deterministic drop conditions

An event can normally be dropped before inference when:

- its `event_id` or retry lineage was already handled;
- it is the agent's own outbound message returning through the channel;
- it is a sibling bot message not addressed to the current agent and no review assignment exists;
- it is a repeated system incident with no changed state;
- it contains only a visible silence marker or acknowledgment;
- the project state version is unchanged and the event is a routine update;

- a public response lease for the task is already held by another agent;
- it is an intermediate tool-progress event that channel policy marks as internal.

6.2.2 Admission decision object

```
{
  "event_id": "evt_01J...",
  "decision": "ADMIT",
  "reason_codes": ["EXPLICITLY_ADDRESSED", "HUMAN_QUESTION"],
  "public_owner": "protocosmo",
  "private_reviewers": ["omegaclaw"],
  "router_action": "NORMAL_MODEL",
  "response_contract_id": "direct_technical_answer_v2",
  "context_profile": "project_compact_plus_evidence",
  "lease_id": "lease_01J..."
}
```

Other decisions include DROP, LOG_ONLY, AGGREGATE, TEMPLATE, and DEFER_UNTIL_COMPLETE.

6.3 Addressing, ownership, and response leases

6.3.1 Ownership precedence and conversational hysteresis

The governor should resolve ownership in the following precedence order:

1. Explicit human address or explicit human override such as `/solo @agent`.
2. Explicit delegation or handoff recorded in task state.
3. Existing active task- or thread-level response lease.
4. Assigned project or task owner.
5. Agent whose message is being directly replied to, when the reply remains within the same task.
6. Capability match:
 - operational, filesystem, code, process, experiment, and tool questions to Protocosmobot;
 - synthesis, critique, philosophy, theory, and long-form explanation to OmegaClaw.
7. Conversational stickiness or recent-speaker bias within a short inactivity window.
8. Channel-specific default spokesperson.
9. Tie-break by current load, confidence, or deterministic precedence.

The active owner should normally remain sticky for the life of a conversational task or for a short timeout such as ten minutes of inactivity. This prevents rapid human dialogue from alternating voices message by message. Stickiness is broken by explicit re-addressing, explicit delegation, task change, capability boundary, lease expiration, owner failure, or a high-severity override.

If the human explicitly requests both agents' independent views, the governor may issue two response leases, but the exception should be deliberate, scoped, logged, and visible. The default scope of an “all agents answer” override is one message, not the entire channel.

6.3.2 Lease semantics

A response lease prevents two agents from independently answering the same event.

```
{
  "lease_id": "lease_01J...",
  "scope": {
    "event_id": "evt_01J...",
    "thread_id": "telegram-thread-9281",
    "task_id": "a6-resweep",
    "state_version": 42
  },
  "holder": "protocosmo",
  "reviewers": ["omegaclaw"],
  "public_slots": 1,
  "expires_at": "2026-07-24T18:47:00Z",
  "status": "ACTIVE"
}
```

Leases should be short-lived and renewable, but normally scoped to the active task or thread rather than only one message. Renewal occurs on a human continuation in the same task or on evidence-bearing progress by the owner. If the owner fails, the lease can expire and be reassigned. A completed lease records the outbound event identifier so retries do not create another answer. Lease heartbeats are internal state updates and never generate public messages.

6.4 Task and thread lifecycle

Threads should have explicit lifecycle state rather than relying on conversational implication.

Suggested states:

OPEN | WAITING_FOR_TOOL | WAITING_FOR_HUMAN | REVIEWING | ANSWERED | CLOSED |
REOPENED

A closed thread should not be reopened by an acknowledgment, repeated status, or sibling echo. Reopening requires one of:

- new evidence;
- a human follow-up;
- a consequential correction;
- a changed project state version;
- an explicit reopen command.

This rule directly prevents “the echo loop is resolved” from becoming the topic of a new echo loop.

6.5 Idempotency and duplicate detection

6.5.1 Exact idempotency

The gateway should record handled event IDs and source message IDs. A retry or reconnect must preserve a `retry_of_event_id` link. Exact duplicates are dropped without model use.

A useful key is:

```
(project_id, task_id, event_type, state_version, intended_recipient)
```

Only one public response should normally be emitted for this key.

6.5.2 Near-duplicate and novelty detection

Exact hashes are insufficient because agents often restate the same content with different wording. However, embeddings and model classifiers are not free. Novelty detection must use a cost-ordered cascade and should call expensive mechanisms only for ambiguous cases.

1. Exact event ID, retry link, source message ID, and normalized payload hash.
2. Structured state checks: task, state version, incident state, requested action, evidence references, and numerical-result changes.
3. Cheap lexical or token-set similarity within a bounded recent thread window.
4. Cached semantic fingerprints or local embeddings when prior stages remain ambiguous.
5. A small model classifier only for high-value cases where an incorrect decision would be costly and deterministic evidence is insufficient.

A provisional feature representation may include:

```
semantic_distance
new_fact_fraction
state_version_changed
new_action_or_decision
new_evidence_reference
new_number_or_result
explicit_correction
severity_and_bypass_flags
```

The first implementation may initialize a weighted score for shadow analysis, but no particular weights should be treated as justified constants. Weights, thresholds, or an alternative decision tree must be calibrated from replay labels and Phase 1 live shadow data. The calibration objective is not maximum suppression; it is high useful-information retention at favorable net cost.

A message with no new fact, no state change, no new action, and no protocol-relevant acknowledgment is a strong suppression candidate even when its wording differs. Conversely, a similar-looking message with a new caveat, changed number, changed evidence, explicit correction, or new requested action must be preserved.

Semantic duplicate detection remains shadow-only until the evaluation set demonstrates a sufficiently low false-silence rate. Deterministic exact control values such as structured `NO_SEND` may be handled earlier.

6.6 Extended smart router

The existing smart-ish router should be extended from model choice to action choice.

Action	Meaning
DROP	Record the event and take no further action. No model call and no outbound message.
LOG_ONLY	Write operational state, incident count, or evidence to logs/project records without public output.
AGGREGATE	Add the event to a daily digest, incident summary, or batch response.
TEMPLATE	Produce a deterministic or parameterized message without an LLM, such as a concise approval request or first incident alert.
CHEAP_MODEL	Use a small or inexpensive model for classification, compression, or simple direct answers.
NORMAL_MODEL	Use the normal capable model for substantive operational or conceptual work.
DEEP_MODEL	Use a high-capability reasoning model for difficult technical analysis, research, theorem work, or explicitly requested depth.

6.6.1 Governor cost cascade and budget

The extended router must account for the cost of governing. It should never spend a model call and hundreds of tokens deciding whether to suppress a tiny acknowledgment that deterministic metadata could classify. Every policy stage records its own CPU time, model calls, tokens, cache hits, and latency.

The preferred rule is:

Use the cheapest decision procedure that can safely resolve the event. Escalate to embeddings or a classifier only when the expected downstream savings and false-silence risk justify the added cost.

The router should enforce a configurable overhead budget and expose the net-efficiency ratio defined in Section 13. A candidate classifier that consumes more resources than it saves should remain disabled even if it is accurate.

The router should also attach a response contract rather than merely a token limit.

```
{
  "message_class": "bot_bot_critique",
  "soft_word_limit": 140,
  "hard_word_limit": 240,
  "required_sections": ["new_objection", "recommended_revision"],
  "forbidden_behaviors": [
    "acknowledgment_only",
    "full_status_recap",
    "new_project_offer"
  ],
  "exceed_soft_limit_when": [
    "safety_critical",
    "multiple_independent_consequences",
    "human_requested_depth"
  ]
}
```

6.7 Private bot-to-bot review bus

Operational coordination should not normally occur as public Telegram conversation. The gateway should provide typed internal messages:

REVIEW_REQUEST | EVIDENCE_REQUEST | TASK_HANDOFF | OBJECTION | DECISION_PROPOSAL

Example:

```
{
  "type": "REVIEW_REQUEST",
  "task_id": "petta-memory-compileadd-gate",
  "from": "protocosmo",
  "to": "omegaclaw",
  "question": "Identify the strongest flaw in the proposed gate.",
  "max_words": 120,
  "public": false,
  "deadline_seconds": 90
}
```

The reviewer returns a compact structured critique. The public owner integrates it into one answer. This retains cross-agent intelligence while eliminating parallel public essays.

6.8 Context construction and project-state snapshots

Each project should maintain a compact durable snapshot. A worker should not need to reread broad chat history on every invocation.

```
{
  "project_id": "omegasim",
  "state_version": 42,
  "current_goal": "Run stratified A6 resweep with CLA diagnostics",
  "last_material_result": "Tight region produced trivial grammars",
  "open_decisions": [],
  "active_blockers": [],
  "next_action": "Run next six unexplored cells",
  "evidence_refs": [
    "PROJECT.md#current-state",
    "runs/a6-stratum_04/report.json"
  ],
  "last_public_reported_version": 39
}
```

The context builder should retrieve:

- the current human message;
- the compact project snapshot;
- a bounded number of relevant evidence excerpts;
- the active response contract;

- optionally a private reviewer response.

It should not include repeated operational logs, old acknowledgments, or entire transcript histories unless specifically necessary.

6.9 Cron workers and update aggregation

Cron and project workers should write state, not directly publish routine updates.

Each worker result should declare:

```
{
  "project_id": "threadkeeper",
  "state_version": 103,
  "material_delta": true,
  "decision_required": false,
  "urgent": false,
  "summary": "Full-batch preflight now rejects malformed late calls atomically.",
  "evidence": ["commit:1ef286a", "tests:355"],
  "next_action": "Extend schema parity tests"
}
```

A single digest publisher runs once per day and includes only projects with meaningful state changes since their last public report. The default human-facing structure is:

1. decisions or approvals needed;
2. blockers and failures;
3. scientific or architectural conclusions that changed;
4. compact one-line deltas for other changed projects;
5. links or pointers to detailed evidence.

Immediate updates bypass the digest only for explicit exceptions:

- human approval required;
- safety, security, authority, or spending threshold reached;
- irrecoverable failure after retry policy;
- material change to scientific conclusion;
- direct human request for live updates;
- time-sensitive blocker where delay would waste substantial resources.

6.10 Incident manager

Repeated operational failures should be represented by one incident object.

```
{
  "incident_id": "inc_provider_503_20260724",
  "class": "MODEL_PROVIDER_UNAVAILABLE",

```

```

"status": "OPEN",
"first_seen": "2026-07-24T08:02:00Z",
"last_seen": "2026-07-24T08:12:00Z",
"occurrence_count": 9,
"diagnosis": "upstream_503",
"policy": "exponential_backoff",
"last_public_notice_state": "OPEN_INITIAL",
"next_public_notice_when": ["RECOVERED", "ESCALATED_30_MIN"]
}

```

The first occurrence may produce a concise alert. Later occurrences update counters silently. A second public message is sent only when diagnosis, severity, required human action, or recovery status changes.

6.11 Probe-before-prose gate

Before allowing a long response about an operational uncertainty, the governor or agent workflow should ask whether a cheap safe test exists. Examples include:

- file readability or existence;
- commit reachability;
- process or cron status;
- pod inventory;
- whether a message explicitly addressed the agent;
- whether a result artifact already exists;
- whether the project state version changed.

A configurable policy might be:

Before generating more than 120 words about an operational uncertainty, perform the cheapest safe discriminating test or state briefly why such a test is unavailable.

This policy belongs partly in agent instructions and partly in workflow code. It should not block conceptual reasoning where no cheap empirical probe exists.

6.12 Egress governor

The egress governor is a final safety net. It should examine every candidate outbound message.

6.12.1 Acknowledgment-like output and structured no-send

Literal NO_REPLY or NO_SEND must be represented as an internal control result and must never be serialized into a human-visible channel. Exact sentinels can therefore be suppressed deterministically once the interface is in place.

Phrases such as the following are only *signals*, not sufficient suppression rules:

```

Noted.
Acknowledged.
Nothing to add.
Staying quiet.

```

```

Already covered.
Not my task.
I will take a look.
Let me inspect.
Continuing...

```

An acknowledgment-like candidate may be suppressed only when all of the following hold:

1. it introduces no new fact, caveat, evidence, or correction;
2. it records no consequential state transition;
3. it requests no action and creates no commitment;
4. it is not required by a safety, authority, cost, ownership, or handoff protocol;
5. the human did not explicitly request acknowledgment.

Consequential acknowledgments should preferably become structured state events such as **ownership_accepted**, **budget_cap_accepted**, **shutdown_confirmed**, or **handoff_accepted**. Channel policy can decide whether such a state transition also deserves a concise visible confirmation.

6.12.2 Semantic firewall for egress processing

The egress governor is a control-plane component, not an unaccountable co-author. It may perform only transformations that are demonstrably non-semantic or mechanically verifiable, including:

- stripping internal control sentinels;
- removing clearly marked tool narration or debug metadata;
- normalizing formatting;
- assembling multipart messages;
- removing byte-identical or paragraph-identical repetition;
- enforcing transport framing and size limits;
- moving detailed evidence to a referenced artifact without deleting the summary or caveat required by the response contract.

It must not silently add, delete, weaken, strengthen, or alter factual claims, recommendations, uncertainty statements, safety caveats, commitments, or requested actions. If a candidate violates a substantive response contract, the governor returns a structured **REGENERATE** request to the originating agent. A small repair model may be used only for a tightly constrained non-semantic transform whose output can be verified against the original.

6.12.3 Candidate outbound schema

```

{
  "lease_id": "lease_01J...",
  "event_id": "evt_01J...",
  "agent_id": "protocosmo",

```

```
"message_class": "direct_technical_answer",
"text": "...",
"claims": ["..."],
"new_facts": ["..."],
"requested_action": null,
"state_version": 42,
"references": ["artifact:report.json"],
"send_intent": true
}
```

The egress governor verifies:

- the sender holds the active lease;
- the thread is not closed without a valid reopen condition;
- the output is not a near-duplicate of recent messages;
- the output contains the required response-contract fields;
- the output does not expose internal-only logs or secrets;
- multipart output is complete before delivery;
- length overruns are justified by policy;
- any egress transformation passes the semantic-firewall policy;
- a substantive contract violation triggers regeneration rather than silent rewriting.

7 Agent-Specific Integration

7.1 Protocosmobot as execution and evidence agent

Protocosmobot should remain the primary agent for:

- code, filesystem, repository, process, and infrastructure work;
- experiments and project workers;
- evidence gathering and verification;
- creating and modifying artifacts;
- operational status and approval requests.

Its tools and skills should return structured internal results by default. Public prose should be generated only after the governor classifies the event as human-visible.

Protocosmobot's local policy should include:

1. Perform ordinary tool sequences silently.
2. Do not narrate each command, retry, file lookup, or provisioning step.

3. Report completion, consequential failure, blocker, or approval request.
4. Store full commands, hashes, tests, and traces in project artifacts or logs.
5. When reporting progress, send only the delta since the last reported state version.
6. Do not interpret another bot's acknowledgment as a new task.
7. When a conceptual synthesis is needed, request a private OmegaClaw review rather than inviting a public second answer.

7.2 OmegaClaw as synthesis and reasoning agent

OmegaClaw should normally receive curated context rather than the raw activity stream. It is well suited for:

- conceptual synthesis;
- criticism and alternative interpretations;
- scientific and philosophical analysis;
- prioritization among options;
- converting structured evidence into a human-readable conclusion;
- long-form writing when requested.

Its local communication policy should include:

1. Do not paraphrase operational updates merely to show comprehension.
2. Do not acknowledge sibling messages unless acknowledgment changes shared state.
3. Do not answer a task owned by Protocosmobot unless assigned as reviewer or explicitly addressed.
4. Return compact private reviews when acting as reviewer.
5. Prefer a conclusion, strongest reasons, uncertainty, and recommendation over a broad essay.
6. Put durable deep treatments into artifacts and post a short pointer in chat.
7. Do not routinely end with an unsolicited proposal to create another formalization, paper, or project.

7.3 When both agents should contribute

Both agents should contribute when the task combines operational evidence and difficult interpretation. Examples include:

- interpreting an experiment result;
- deciding whether a security hardening change is sufficient;
- evaluating a proof-engine report;
- analyzing a failed training run and choosing the next experiment;

- preparing a strategic daily digest from many project workers.

The interaction should be private and bounded:

1. Protocosmobot gathers evidence and proposes a factual summary.
2. OmegaClaw receives a targeted review question with a short word budget.
3. Protocosmobot or the designated spokesperson integrates the critique.
4. One public response is sent.

The human can explicitly request independent answers when diversity itself is valuable.

8 Algorithms and Decision Procedures

8.1 Inbound event processing

```
def process_inbound(event):
    event = normalize(event)

    if exact_duplicate(event):
        return Decision.drop("EXACT_DUPLICATE")

    if is_own_message_loopback(event):
        return Decision.drop("OWN_MESSAGE_LOOPBACK")

    if incomplete_multipart(event):
        buffer(event)
        return Decision.defer("WAITING_FOR_PARTS")

    if update_existing_incident(event):
        if incident_requires_public_transition(event):
            return incident_notice_decision(event)
        return Decision.log_only("INCIDENT_REPEAT")

    if routine_project_update(event):
        if not event.material_delta:
            return Decision.log_only("NO_MATERIAL_DELTA")
        if not immediate_update_exception(event):
            aggregate_into_daily_digest(event)
            return Decision.aggregate("DAILY_DIGEST")

    owner = resolve_public_owner(event)

    if owner is None:
        return Decision.drop("NO_ELIGIBLE_RESPONDER")

    if active_lease_conflicts(event, owner):
        return Decision.drop("RESPONSE_ALREADY_CLAIMED")

    lease = issue_lease(event, owner)
    reviewers = choose_private_reviewers(event, owner)
    route = choose_router_action(event, owner)
    contract = choose_response_contract(event, route)
```

```

context = build_compact_context(event, owner, contract)

return Decision.admit(
    owner=owner,
    reviewers=reviewers,
    route=route,
    contract=contract,
    context=context,
    lease=lease,
)

```

8.2 Public owner resolution

```

def resolve_public_owner(event, now):
    if override := explicit_human_override(event):
        return override.owner

    if event.addressing.explicit_agents:
        return highest_priority_explicit_agent(event)

    if delegated := lookup_explicit_delegation(event.scope.task_id):
        return delegated

    if lease := active_thread_or_task_lease(event, now):
        if not lease_break_condition(event, lease):
            return lease.holder

    if task_owner := lookup_task_owner(event.scope.task_id):
        return task_owner

    if event.addressing.reply_to_agent and same_task(event):
        return event.addressing.reply_to_agent

    scores = {
        "protocosmo": operational_fit(event),
        "omegaclaw": synthesis_fit(event),
    }
    if clear_capability_winner(scores):
        return deterministic_argmax(scores)

    if sticky := recent_thread_owner(event, timeout_minutes=10):
        return sticky

    if spokesperson := channel_spokesperson(event.source.channel_id):
        return spokesperson

    return deterministic_argmax(scores)

```

8.3 Egress processing

```

def process_egress(candidate):
    if not valid_lease(candidate):
        return Suppress("NO_ACTIVE_LEASE")

```

```

if no_send_semantics(candidate.text):
    complete_lease_without_send(candidate.lease_id)
    return Suppress("NO_REPLY_SEMANTICS")

if contains_internal_tool_narration(candidate):
    candidate = remove_or_summarize_tool_narration(candidate)

if semantic_duplicate_of_recent(candidate):
    complete_lease_without_send(candidate.lease_id)
    return Suppress("SEMANTIC_DUPLICATE")

if not satisfies_contract(candidate):
    if only_nonsemantic_repair_needed(candidate):
        candidate = apply_verified_nonsemantic_transform(candidate)
    else:
        return Regenerate(
            reason="SUBSTANTIVE_CONTRACT_VIOLATION",
            preserve=required_semantic_fields(candidate.contract),
        )

if incomplete_multipart(candidate):
    buffer(candidate)
    return Suppress("WAITING_FOR_OUTPUT_PARTS")

outbound_id = send(candidate)
record_public_response(candidate, outbound_id)
complete_lease(candidate.lease_id, outbound_id)
return Sent(outbound_id)

```

8.4 Daily digest construction

```

def build_daily_digest(project_states):
    changed = [
        p for p in project_states
        if p.state_version > p.last_public_reported_version
        and p.material_delta
    ]

    decisions = [p for p in changed if p.decision_required]
    blockers = [p for p in changed if p.active_blockers]
    conclusions = [p for p in changed if p.conclusion_changed]
    routine = [
        p for p in changed
        if p not in decisions + blockers + conclusions
    ]

    return render_digest(
        decisions=decisions,
        blockers=blockers,
        conclusions=conclusions,
        routine=routine,
        max_words=900,
    )

```

9 Response Classes and Progressive Disclosure

The system should use response classes rather than one universal style.

Class	Default length	Expected content
Receipt or acknowledgment	0 words	Hidden receipt, state transition, or reaction. No LLM call where possible.
Routine project delta	60–100 words	Changed result, evidence pointer, next action, blocker if any.
Bot-to-bot proposal	100–160 words	One proposal, rationale, explicit question.
Bot-to-bot critique	100–160 words	Strongest objection, correction, and recommended revision.
Human decision request	80–160 words	Decision needed, options, recommendation, consequence of delay.
Direct substantive answer	200–400 words	Conclusion, strongest reasons, uncertainty, recommended next step.
Deep requested analysis	Flexible	Full treatment, preferably stored as a durable artifact with compact chat summary.
Operational incident	40–100 words	What failed, current impact, retry policy, whether human action is needed.

Progressive disclosure should be targeted. Instead of “would you like more detail?”, the response may expose named expansion handles:

```
DETAIL: statistical caveat
DETAIL: implementation architecture
EVIDENCE: test results
ALTERNATIVE: conservative design
```

A follow-up request can retrieve only the named detail. This reduces the tendency of broad clarification prompts to trigger another general essay.

10 Channel Policies

Every channel should have a policy object. The current Telegram rooms and future OmegaHive channels may differ substantially.

```
{
  "channel_id": "telegram:protobots",
  "channel_type": "mixed_human_ai",
  "visibility": "human_visible",
  "default_spokesperson": "protocosmo",
  "allowed_public_responders": 1,
  "bot_bot_public_discussion": "exception_only",
  "tool_narration": "forbidden",
  "routine_update_policy": "daily_digest",
  "incident_policy": "coalesced",
  "max_bot_turns_per_state_version": 2,
  "retention": "full_log",
  "privacy_label": "private_group"
```

}

Recommended channel types include:

- Human-direct** A human converses with one selected agent. Other agents participate only through private review.
- Mixed human/AI project room** Humans and agents share a room. One project spokesperson answers by default.
- AI-only coordination room** Structured task handoffs, review requests, and evidence exchange. Logged but not sent to social media.
- AI-only deliberation room** A bounded committee explores alternatives and returns one synthesis artifact.
- Public social channel** Strongest egress policy, moderation, privacy filtering, and designated spokesperson.
- Operational log stream** Machine-readable events; not treated as conversational input unless an incident transition occurs.

11 Human Override Grammar

The human operator needs explicit, memorable controls that are interpreted as first-class channel policy inputs rather than vague natural-language hints. Natural-language equivalents may be recognized, but slash commands have deterministic semantics and are always logged.

Command	Default meaning
/all-answer	Permit all eligible agents to answer the current message independently. Scope defaults to one message.
/solo @agent	Assign one public responder for the current message or stated scope.
/review @agent	Request a private critique from the named agent while preserving one public owner.
/quiet	Suppress nonurgent autonomous updates for the stated scope. Safety and explicit decision requests still bypass.
/verbose	Permit a detailed response for the current thread without weakening ownership or safety policy.
/brief	Apply a concise response contract for the current message or thread.
/show-suppressed	Display or summarize recent shadow or actively suppressed decisions, subject to privacy policy.
/reopen	Reopen a closed task or thread.
/status	Request compact current state, owner, blockers, and next action.

Every override has an explicit scope: current message, current thread, project, channel, or until revoked. Commands that could substantially increase cost or public traffic should default to the narrowest scope. The decision ledger records the command, authenticated issuer, scope, expiration, and policy effects.

12 Failure Modes and Safety Considerations

12.1 False silence

The largest risk is suppressing a message that contained a useful warning or correction. Mitigations include:

- shadow-mode deployment;
- conservative deterministic suppression first;
- bypass for safety, security, authority, cost, and explicit human addressing;
- logging every suppression decision with reason codes;
- easy human command to inspect suppressed events;
- periodic audit samples reviewed by a strong model or human.

12.2 Over-centralization

A shared governor can become a bottleneck or single point of failure. The design should therefore:

- keep the governor policy-driven and relatively simple;
- preserve direct emergency paths for high-severity agents;
- use durable event logs so decisions can be replayed;
- allow local channel governors in a larger hive;
- avoid embedding substantive domain reasoning in the governor.

12.3 Lease deadlock or abandonment

An agent may acquire a lease and fail. Leases need timeouts, heartbeats only while real work is underway, and deterministic reassignment. A lease expiration should not itself generate public chatter.

12.4 Semantic deduplication errors

Two messages may be semantically similar but differ in an important caveat. Early semantic suppression should therefore operate in shadow mode or require strong confidence. New numerical evidence, changed state version, changed requested action, explicit correction, or high-severity flags should override semantic similarity.

12.5 Private review leakage

AI-only channels may contain speculative, sensitive, or low-confidence material. Egress must enforce privacy labels and ensure that public synthesis does not expose internal-only content without authorization.

12.6 Human confusion about who answered

The public response should clearly retain the identity of the designated agent. Internal reviewers can be credited when useful, but the system should not blur agent identity or create the appearance that one bot performed another bot's tools.

12.7 Governor overhead and negative savings

The governor can become wasteful if it routinely invokes embeddings or classifiers for trivial events. Mitigation requires a cost-ordered decision cascade, cache reuse, per-stage metrics, and an explicit net-savings gate. Expensive novelty logic remains off when deterministic metadata resolves the event or when projected savings do not exceed overhead by the configured margin.

12.8 Ownership thrashing

Per-event owner resolution may alternate voices during rapid dialogue. Thread- and task-level hysteresis, explicit lease-break rules, and the ownership precedence order prevent disjointed conversation. Stickiness must not override explicit human address, delegation, capability boundaries, or high-severity intervention.

12.9 Semantic mutation by the governor

An egress repair model could quietly change the originating agent’s meaning. The semantic firewall prohibits substantive rewriting. Non-semantic transforms must be verifiable; otherwise the candidate is returned for regeneration. Audit records preserve the original candidate, transformation type, and final output.

13 Observability and Evaluation

The rollout should be measured, not judged only by subjective quietness.

13.1 Core metrics

Metric	Interpretation
LLM calls per human-visible useful response	Direct measure of coordination overhead.
Tokens per material project delta	Measures whether routine work is communicated efficiently.
No-op invocation rate	Fraction of LLM calls that result in acknowledgment, silence marker, or no new content.
Duplicate outbound rate	Fraction of public messages semantically duplicating a recent message.
Parallel-response rate	Fraction of human events receiving more than one unrequested public agent answer.
Tool-narration rate	Fraction of public messages primarily describing intermediate execution steps.
Incident amplification factor	Public incident messages divided by distinct incident state transitions.
False-silence rate	Suppressed events later judged to have contained useful or necessary content, separated into missed request, correction, blocker, approval, safety/cost alert, state transition, and minority view.
Governor model-call rate	Fraction of events for which the governor itself invokes a model or embedding service.
Governor tokens and compute cost	Direct operating cost of admission, classification, and repair logic.

Metric	Interpretation
Governor-added latency	Added p50 and p95 latency from control-plane processing.
Net cost avoided	Estimated downstream inference cost avoided minus governor operating cost.
Governor efficiency ratio	Avoided downstream inference cost divided by governor operating cost.
Human clarification burden	Number of times the human must ask what is actually happening after agent updates.
Time to decision	Elapsed time from decision-relevant event to clear human decision request or answer.
Useful-information retention	Human or reviewer rating of whether important caveats and options remain available.

13.2 Predeclared activation and go/no-go gates

Before Phase 0 data are inspected, define the decision criteria for each active feature. The exact numerical thresholds may be adjusted to the system's traffic volume, but they should be declared before activation to avoid post-hoc justification.

At minimum, activation requires:

- a material measured no-op, duplicate, incident-amplification, or parallel-response problem for the feature being enabled;
- projected avoided cost substantially greater than governor overhead;
- replay and live shadow estimates of false silence below the approved threshold for the relevant event class;
- no material degradation in time to useful answer or human-rated useful-information retention;
- explicit human approval for the feature flag and channel scope.

A feature can be rejected while simpler features proceed. For example, semantic embedding-based suppression may fail its economics gate even though structured NO_SEND, own-message filtering, incident coalescing, and response leases remain strongly justified.

13.3 Audit records

Every governor decision should be recorded:

```
{
  "event_id": "evt_01J...",
  "decision": "DROP",
  "reason_codes": ["ACK_ONLY", "THREAD_CLOSED"],
  "policy_version": "ocg-0.3.1",
  "candidate_owner": "omegaclaw",
  "model_call_avoided": true,
  "estimated_tokens_avoided": 850,
  "review_sample_bucket": "weekly_1_percent"
}
```

This supports rollback, policy tuning, and scientific analysis of hive communication behavior.

14 Deployment and Migration Strategy

14.1 Phase 0: baseline instrumentation only

Add event IDs, origin metadata, model-call counters, token estimates, outbound-message fingerprints, governor-stage timing, and channel/task ownership observations without changing dispatch, delivery, suppression, or topology. Instrumentation taps and durable logs are allowed; live rerouting and public-output changes are not.

Before reviewing the results, record the go/no-go criteria for each later feature. Phase 0 must run long enough and collect enough varied events to cover direct human questions, bot-bot exchanges, cron updates, errors, attachments, approval requests, urgent alerts, and long-running tasks.

14.2 Phase 1: shadow decisions and false-silence audit

Run admission, ownership, incident, response-contract, semantic-firewall, and egress decisions in parallel with current behavior. Record what would have been suppressed, aggregated, rerouted, privately reviewed, or regenerated. No shadow decision changes production behavior.

Audit false-silence candidates by category, calibrate novelty features and thresholds, quantify governor overhead, and estimate net savings. The shadow period should be bounded by both elapsed time and minimum event coverage, not merely one calendar day.

14.3 Activation Gate 1: explicit approval

At the end of Phase 1, produce a decision report containing baseline rates, shadow confusion matrices, false-silence examples, governor overhead, projected net savings, topology finding, and proposed feature scopes. No live suppression, lease enforcement, topology change, or semantic novelty activation occurs until the human operator explicitly approves the named flags and channel scope.

14.4 Phase 2: deterministic suppression

Activate only approved, high-confidence deterministic classes:

- suppress exact structured `NO_SEND` control results;
- block own-message loopback and exact retry duplicates;
- suppress acknowledgment-like prose only when structured novelty and protocol-state checks confirm no consequential transition;
- coalesce exact repeated incidents;
- stop public tool-step narration from known worker event types;
- aggregate “no material change” project updates.

Maintain a one-switch rollback and a human-visible suppression-inspection command.

14.5 Phase 3: shared response leases with hysteresis

Enable a single public owner per task or thread with conversational hysteresis. Begin with mixed human/AI project channels. Preserve explicit multi-agent requests and verify that rapid human dialogue does not cause owner thrashing.

14.6 Phase 4: private review bus

Move routine bot-to-bot critiques and evidence requests off Telegram. Bound reviews by question, time, and response length.

14.7 Phase 5: daily digest and compact project state

Migrate cron workers to state-writing mode and deploy one digest publisher. Use compact project snapshots for context construction.

14.8 Phase 6: calibrated semantic novelty and adaptive budgets

After sufficient labeled data are collected, compare calibrated semantic scoring, a decision tree, and a deterministic-only baseline. Activate near-duplicate suppression only if the false-silence and net-economics gates pass. Retain the deterministic-only configuration as a supported endpoint. Adapt word and model budgets based on message class, project risk, and human preferences.

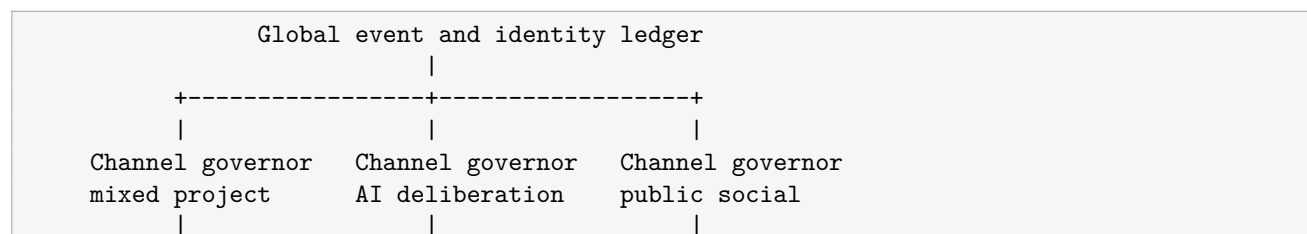
15 Extension to a Richer OmegaHive

15.1 From a two-agent governor to a communication fabric

A larger OmegaHive may contain many specialized agents, such as coding, mathematical, scientific, communications, governance, safety, memory, project-management, and social agents. It may also contain many channel types:

- AI-only task and review channels that are fully logged but not placed on Telegram, Slack, or another social platform;
- mixed human/AI project rooms resembling the current Telegram channels;
- human-only governance or decision rooms;
- public or semi-public social channels;
- machine-only operational event streams;
- temporary deliberation rooms created for one problem and closed afterward.

At this scale, a single central governor may become too coarse. The recommended architecture is federated.



local agents	review committee	spokespersons
--------------	------------------	---------------

Each channel governor enforces local policy. The global ledger provides event identity, agent identity, project state versions, privacy labels, task ownership, and cross-channel deduplication.

15.2 Typed channels and boundary brokers

Every channel should declare:

- participants and eligible agent roles;
- human visibility;
- privacy and retention class;
- default spokesperson;
- maximum public responders;
- whether bot-to-bot free discussion is allowed;
- response and token budgets;
- escalation paths;
- whether content may cross into another channel.

A **boundary broker** governs movement from internal AI-only channels into mixed or public channels. Internal deliberation may be broad and exploratory, but the boundary broker produces a concise, provenance-linked synthesis and enforces privacy constraints.

15.3 Topic and role subscriptions

Agents should not ingest every message. They subscribe by role, project, topic, and event type.

Example subscriptions:

```
{
  "agent_id": "mathmaxxer",
  "subscriptions": [
    {"project": "rh", "event_types": ["proof_target", "counterexample"]},
    {"role": "formal_reviewer", "severity": ["high"]}
  ],
  "exclude": ["routine_build_log", "acknowledgment", "social_banter"]
}
```

This prevents large-agent hives from becoming all-to-all broadcast networks.

15.4 Bounded deliberation committees

When diversity is useful, the governor creates a temporary committee with explicit roles:

- proposer;
- adversarial critic;

- evidence checker;
- synthesizer;
- optional safety or governance reviewer.

The committee operates in an AI-only logged room with a bounded number of turns. It outputs one decision artifact and one human-facing summary. This is preferable to allowing every agent to answer independently in a mixed channel.

15.5 Attention budgets

Each project, channel, and task can carry a communication budget:

```
{
  "task_id": "omegasim-a7-design",
  "max_agent_turns": 8,
  "max_deep_model_calls": 2,
  "max_total_output_tokens": 7000,
  "max_public_messages": 2,
  "deadline": "2026-07-26T18:00:00Z"
}
```

Agents may request a budget extension with a concrete justification. This makes thoughtful exceptions possible without allowing unbounded conversational drift.

15.6 Cross-channel state and duplicate prevention

The same project may appear in an AI-only research room, a mixed human project channel, and a public update channel. The global event ledger should ensure that one result is not independently summarized by many agents in every room.

A result event can have channel-specific publication states:

```
{
  "result_id": "res_omegasim_0042",
  "internal_research": "FULL_ARTIFACT_AVAILABLE",
  "mixed_project": "SUMMARY_PUBLISHED",
  "public_social": "NOT_ELIGIBLE",
  "daily_digest": "INCLUDED"
}
```

15.7 Reputation and confidence without chatter

A richer hive may use agent reputation or confidence to choose owners and reviewers. These values should affect routing silently rather than being continuously discussed in-channel. For example, a high-reliability coding agent may receive operational ownership while a high-quality critic is assigned privately.

15.8 Governance, moderation, and social behavior

Mixed and public channels need stronger policy than AI-only rooms:

- designated spokesagents;

- privacy redaction and secret scanning;
- rate limits and cooldowns;
- explicit distinction between agent speculation and project commitments;
- human override and correction paths;
- moderation of conflict escalation and repetitive bickering;
- provenance links for consequential claims.

The aim is not to remove personality or spontaneous social dynamics. It is to ensure that such dynamics are intentional features of a social room rather than accidental consequences of system-event routing.

16 Recommended Initial Configuration for the Current Proto-Hive

A practical starting configuration is:

Setting	Recommended value
Default public responder	Explicitly addressed agent; otherwise project owner; otherwise Protocosmobot for operational content and OmegaClaw for conceptual content.
Maximum public responders	1, unless the human explicitly requests independent views.
Private reviewer	Optional one-agent review for difficult or consequential tasks.
Routine project updates	Once daily, only for material deltas.
Tool narration	Internal only.
Visible <code>NO_REPLY</code>	Hard-suppressed.
Acknowledgments	Hidden receipts or reactions; no LLM call when possible.
Repeated incidents	One open notice, silent occurrence count, one recovery or escalation notice.
Bot-to-bot public turns	Maximum two substantive turns per state version.
Long analysis	Chat summary plus durable artifact.
Operational uncertainty	Cheap probe before response above 120 words.
Semantic duplicate suppression	Shadow mode initially; active only after calibration, economics review, and explicit approval.
Ownership hysteresis	Retain task/thread owner for ten minutes of inactivity unless a higher-precedence rule breaks the lease.
Egress rewriting	Non-semantic transforms only; substantive violations return to the originating agent for regeneration.
Human overrides	Deterministic slash commands with narrow default scope.
Governor overhead	Measured and budgeted; expensive classification disabled when net savings are unfavorable.

17 Conclusion

The present proto-hive does not need less intelligence. It needs a communication substrate that makes intelligence selective about when to speak, who should speak, and what information belongs in public conversation.

The highest-value change is to move response admission, ownership, idempotency, update aggregation, and no-op suppression into a shared gateway-level governor. Agent prompts and length limits remain useful, but they become the final layer of refinement rather than the sole mechanism of control.

The architecture preserves the useful asymmetry between Protocosmobot and OmegaClaw: one gathers evidence and acts; the other synthesizes and critiques. Their collaboration becomes private, targeted, and bounded, while humans normally receive one coherent answer. This conclusion does not depend on a single physical deployment topology: centralized and coordinated-local placements share the same policy and ledger semantics. The governor itself remains cost-bounded, shadow-tested, explicitly activated, and barred from silently rewriting substantive cognition-plane content. Routine project activity becomes durable state plus one daily digest. Errors become incidents rather than repeated conversational events. Silence becomes a real action. Deep reasoning remains available when it is genuinely useful.

The same design scales naturally to a larger OmegaHive by federating channel governors around a shared identity and event ledger, separating internal AI deliberation from human-visible output, and using explicit spokesagents, response leases, attention budgets, and boundary brokers.

A Detailed Implementation Plan for an OpenClaw Coding Agent

This appendix is written as an actionable plan for an OpenClaw coding agent. The agent should execute it incrementally, preserve current functionality, and provide evidence for every gate.

A.1 Implementation mandate

Implement a shared OmegaHive Conversation Governor around the existing OpenClaw gateway used by Protocosmobot and OmegaClaw. The implementation must reduce duplicate and no-op model calls while preserving explicit human addressing, safety-critical intervention, deep requested analysis, and current tool functionality.

Do not begin by rewriting the agents. Begin by discovering the actual event flow and inserting observability. Treat the shared pre-inference boundary as an empirical question and produce centralized, coordinated-local, or weak-fallback placement recommendations. Use small commits, provider-free tests where possible, and a shadow-mode rollout. No active suppression or topology-changing behavior is permitted until the Phase 1 decision report is explicitly approved.

A.2 Operating constraints for the implementing agent

1. Do not modify live routing until a provider-free replay harness exists.
2. Do not suppress messages in production until shadow logs have been reviewed.
3. Do not publish raw user transcripts or private artifacts to public repositories.
4. Preserve current model-router behavior behind a feature flag.
5. Keep policy configuration external to core code where practical.
6. Use deterministic checks before model-based classification.
7. Record exact commands, tests, commits, and rollback instructions.
8. Avoid narrating every implementation step to the human; report milestone completion, blockers, and decisions only.
9. Do not use the egress governor to alter substantive claims; implement the semantic firewall before any repair path.
10. Measure governor cost and latency from the first shadow implementation.
11. Treat novelty weights and thresholds as calibration parameters, not fixed design truths.

A.3 Phase A: repository and runtime discovery

A.3.1 Tasks

1. Identify the repository or repositories containing:
 - OpenClaw gateway ingress;
 - Telegram adapter;
 - agent dispatch;

- existing smart router;
 - model invocation wrappers;
 - cron/project worker publication paths;
 - OmegaClaw gateway client;
 - message persistence or thread state.
2. Produce an event-flow map from inbound Telegram update to outbound send.
 3. Identify every place where an LLM call can occur before a public response.
 4. Identify whether both agents share one process, one host, one queue, or only an inference endpoint.
 5. Identify current message IDs, reply metadata, bot sender IDs, channel IDs, and any existing idempotency keys.
 6. Locate current configuration and feature-flag mechanisms.
 7. Determine whether Topology A, B, or C is feasible and identify the exact atomic lease boundary available in each feasible option.
 8. Estimate whether OmegaClaw requests traverse the OpenClaw gateway before or only at model inference time.

A.3.2 Deliverable

Create `docs/conversation_governor/current_event_flow.md` containing:

- actual module and function names;
- an ASCII flow diagram;
- proposed insertion points for ingress and egress governors;
- risks and unknowns;
- confirmation of whether a shared pre-inference boundary exists;
- a recommended topology plus fallback topology;
- expected savings and limitations of each feasible placement.

A.3.3 Acceptance criteria

- At least one real inbound event is traced end to end using logs or tests.
- The exact location where model selection occurs is identified.
- The exact location where Telegram sends occur is identified.
- No live behavior is changed.
- The topology claim is supported by at least one traced event from each agent.

A.4 Phase B: event schema and durable ledger

A.4.1 Tasks

1. Implement a canonical `EventEnvelope` data structure.
2. Add stable event IDs and preserve source message IDs.
3. Add origin chain, retry lineage, sender kind, explicit mentions, reply-to agent, channel type, project ID, task ID, thread ID, and state version fields.
4. Implement a lightweight durable store, initially SQLite or the project's existing database.
5. Add tables or collections for:
 - events;
 - governor decisions;
 - response leases;
 - thread state;
 - project report state;
 - incidents;
 - recent semantic fingerprints.
6. Ensure that storage failures fail open for conversational delivery during early rollout, while logging the failure.

A.4.2 Suggested module layout

Adapt names to the repository, but aim for a separation like:

```
conversation_governor/  
  __init__.py  
  models.py  
  event_normalizer.py  
  ledger.py  
  admission.py  
  ownership.py  
  leases.py  
  incidents.py  
  digests.py  
  context_builder.py  
  egress.py  
  policies.py  
  semantic_dedup.py  
  overrides.py  
  semantic_firewall.py  
  calibration.py  
  metrics.py  
  feature_flags.py
```

A.4.3 Acceptance criteria

- Event envelopes round-trip through the store deterministically.
- Retrieved source messages can be linked to their original events.
- Bot-originated outbound messages retain origin metadata when re-ingested.
- Provider-free unit tests cover malformed envelopes and storage recovery.

A.5 Phase C: transcript-derived replay corpus

A.5.1 Tasks

Construct a private test corpus from the supplied transcript patterns. Do not publish the raw conversations. Convert them into minimal synthetic cases representing:

1. acknowledgment loop;
2. visible NO_REPLY;
3. duplicate extended analysis;
4. repeated 503 traceback;
5. tool-step narration during provisioning;
6. two agents answering one human message;
7. message addressed to one agent but answered by another;
8. multipart message misinterpreted before completion;
9. “no material change” project update;
10. file-access speculation resolved by a cheap probe;
11. explicit request for both agents’ independent opinions;
12. high-severity correction from a non-owner agent;
13. acknowledgment that records task ownership or budget acceptance;
14. rapid same-thread dialogue requiring ownership hysteresis;
15. egress candidate whose shortening would remove a safety caveat;
16. explicit `/all-answer`, `/solo`, and `/quiet` overrides;
17. governor classifier whose cost exceeds the downstream message it would save.

Represent each case as input events plus expected governor decisions.

A.5.2 Example fixture

```
{
  "name": "ack_loop_suppression",
  "events": [
    {"sender": "protocosmo", "text": "Work complete. NO_REPLY"},
    {"sender": "omegaclaw", "text": "Noted that work is complete."}
  ],
  "expected": [
    {"decision": "DROP", "reason": "NO_REPLY_SEMANTICS"},
    {"decision": "DROP", "reason": "ACK_ONLY"}
  ]
}
```

A.5.3 Acceptance criteria

- At least eighteen replay scenarios exist.
- They run without provider calls.
- Each scenario asserts both the decision and the absence or presence of outbound sends.
- The replay runner can compare legacy behavior with shadow governor behavior.

A.6 Phase D: shadow admission engine

A.6.1 Tasks

1. Implement deterministic rules for exact duplicates, own-message loopback, explicit addressing, repeated incidents, routine unchanged updates, and visible no-reply phrases.
2. Implement owner resolution without enforcing it.
3. Add decision reason codes.
4. Invoke the shadow governor on every inbound event while leaving legacy dispatch unchanged.
5. Record estimated downstream model calls and tokens that would have been avoided.
6. Record governor CPU time, latency, model calls, embedding calls, and tokens.
7. Emit shadow decisions for ownership hysteresis, human overrides, acknowledgment state changes, and semantic-firewall violations.
8. Add a daily internal shadow report and a final activation decision report.

A.6.2 Acceptance criteria

- Shadow mode cannot suppress or reroute production messages.
- Every decision is auditable by event ID.
- Replay corpus passes.
- The shadow run satisfies the predeclared minimum time and event-coverage requirement.

- The final report includes false-silence categories, confusion examples, overhead, projected net savings, and a feature-by-feature go/no-go recommendation.
- No active feature can be enabled without explicit human approval recorded in the ledger.

A.7 Phase E: hard no-op suppression

Activate only deterministic egress suppression behind a feature flag.

A.7.1 Initial suppression classes

- exact NO_REPLY;
- exact or near-exact acknowledgment-like prose only when it carries no new claim, evidence, action, commitment, or consequential protocol state;
- “staying quiet,” “nothing to add,” and “already covered” under the same structured no-delta test;
- own outbound message loopback;
- exact repeated system-event payloads;
- known intermediate tool-progress event types.

A.7.2 Required safeguards

- Never suppress explicit human messages.
- Never suppress safety, security, cost, authority, or irreversible-action alerts.
- Log candidate text and reason privately.
- Provide a command to inspect recent suppressed messages.
- Provide a single feature flag to disable all active suppression.
- Preserve acknowledgments that accept ownership, a budget or safety constraint, a handoff, shutdown, or another consequential state transition.
- Apply no substantive egress rewriting; contract failures regenerate at the source agent.

A.7.3 Acceptance criteria

- Replay corpus shows no regression in explicit human answers.
- Live canary produces no known false silence for the approved observation window.
- Token and message counts decline measurably after governor overhead is deducted.
- Human override commands and suppression-inspection commands work in the canary channel.

A.8 Phase F: response leases and ownership

A.8.1 Tasks

1. Implement lease creation, renewal, completion, expiration, and reassignment.

2. Integrate explicit mentions, reply-to metadata, project ownership, and role fit.
3. Require a valid lease before public egress from either agent.
4. Allow a policy exception for explicit human requests for multiple independent answers.
5. Add a conflict log when two agents attempt to acquire the same lease.
6. Scope leases to tasks or threads, add a configurable stickiness timeout, and implement explicit break conditions.
7. Implement the ownership precedence order and verify that recent-speaker bias never defeats explicit address or capability boundaries.

A.8.2 Acceptance criteria

- One human event produces one public answer in default mode.
- Explicit dual-agent requests produce two answers or one labeled comparative synthesis, according to policy.
- Lease expiration recovers from a crashed owner without duplicate sends.
- Retries after completed leases do not resend.
- Rapid follow-up dialogue retains a coherent public owner unless a documented break condition occurs.

A.9 Phase G: human overrides and semantic egress firewall

A.9.1 Tasks

1. Implement the slash-command grammar and scoped override records.
2. Authenticate the human issuer and record override scope and expiration.
3. Implement deterministic precedence between overrides, leases, channel policy, and high-severity bypasses.
4. Implement a semantic-firewall policy that classifies allowed non-semantic transforms.
5. Replace any generic egress repair call with either a verified transform or structured regeneration request.
6. Preserve original candidate text, transform metadata, and final output in private audit logs.

A.9.2 Acceptance criteria

- `/all-answer`, `/solo`, `/review`, `/quiet`, `/brief`, `/verbose`, `/show-suppressed`, `/reopen`, and `/status` pass replay tests.
- Overrides default to one-message scope unless the user specifies otherwise.
- A formatting-only violation can be repaired without claim changes.

- A candidate whose shortening would remove a caveat is returned for regeneration, not rewritten by the governor.
- Audit comparison shows that no substantive proposition was added or removed by allowed transforms.

A.10 Phase H: private review bus

A.10.1 Tasks

1. Implement typed private review requests through the gateway.
2. Add bounded reviewer prompts and structured review returns.
3. Ensure private reviews are not automatically posted to Telegram.
4. Permit the public owner to include or ignore reviewer feedback.
5. Record reviewer provenance in internal logs and optionally in public artifact metadata.

A.10.2 Acceptance criteria

- A test task uses Protocosmobot as owner and OmegaClaw as private reviewer.
- Only one public response is emitted.
- The public response reflects at least one reviewer correction in the test fixture.
- Reviewer timeout does not block the owner indefinitely.

A.11 Phase I: project snapshots and digest migration

A.11.1 Tasks

1. Define `ProjectStateSnapshot` and state-version semantics.
2. Modify one low-risk cron worker to write state instead of sending Telegram updates.
3. Implement a daily digest generator.
4. Add immediate-update exception flags.
5. Migrate workers gradually, one project at a time.
6. Track `last_public_reported_version` per project.

A.11.2 Acceptance criteria

- The pilot project emits at most one routine daily update.
- Multiple same-day commits are summarized as one delta.
- Approval-required events still notify immediately.
- “No material change” creates no public message.

A.12 Phase J: incident coalescing

A.12.1 Tasks

1. Define incident classes and state transitions.
2. Convert provider failures, cron failures, and repeated tool errors into incident updates.
3. Send first-open, escalation, required-action, and recovery notices only.
4. Store raw traces in logs with secure access.

A.12.2 Acceptance criteria

- Ten repeated identical 503 events produce one initial notice and one recovery notice.
- A changed diagnosis produces a new notice.
- Raw trace remains retrievable for debugging.

A.13 Phase K: semantic deduplication and calibration

A.13.1 Tasks

1. Add semantic fingerprints using an existing embedding service or inexpensive local model.
2. Compare candidates only within bounded recent task/thread windows.
3. Extract simple structured novelty indicators: new numbers, new evidence refs, changed state version, changed requested action, correction markers.
4. Run in shadow mode and label false-positive and false-negative cases by event class.
5. Compare a calibrated score, decision tree, and deterministic-only baseline.
6. Measure embedding/classifier overhead and calculate net savings.
7. Activate only above an approved confidence threshold and only if the economics gate passes.

A.13.2 Acceptance criteria

- Exact paraphrases are reliably detected.
- New caveats or changed results are not suppressed in the evaluation set.
- High-severity and explicit-human-follow-up messages bypass semantic suppression.
- The selected policy outperforms the deterministic-only baseline on useful-information retention without violating the overhead budget.
- Fixed example weights are not shipped unless calibrated evidence specifically supports them.

A.14 Phase L: metrics dashboard and policy tuning

Implement counters and reports for:

- model calls avoided;
- estimated input and output tokens avoided;
- suppressed no-op messages;
- duplicate public responses prevented;
- incident repeats coalesced;
- daily digest compression ratio;
- false-silence reports;
- average time to useful answer;
- response ownership distribution between agents;
- ownership-switch and thrash rate;
- governor CPU, latency, model, embedding, and token overhead;
- net cost avoided and efficiency ratio;
- semantic-firewall regeneration and transform counts;
- override-command usage and scope.

Do not optimize only for token reduction. Establish a utility audit that samples suppressed and compressed events.

A.15 Feature flags

At minimum, provide:

```
OCG_SHADOW_MODE
OCG_ACTIVE_EGRESS_SUPPRESSION
OCG_RESPONSE_LEASES
OCG_PRIVATE_REVIEW_BUS
OCG_DAILY_DIGEST
OCG_INCIDENT_COALESCING
OCG_SEMANTIC_DEDUP
OCG_COMPACT_CONTEXT
OCG_HUMAN_OVERRIDES
OCG_OWNERSHIP_HYSTERESIS
OCG_SEMANTIC_FIREWALL
OCG_GOVERNOR_COST_BUDGET
OCG_TOPOLOGY_MODE
```

Feature flags should be independently reversible.

A.16 Testing strategy

A.16.1 Unit tests

Test schemas, reason codes, exact deduplication, mention parsing, lease transitions, thread reopening, incident transitions, and egress suppression.

A.16.2 Property tests

Useful properties include:

- one active public lease per default event scope;
- completed lease plus retry never yields a second send;
- explicit human addressing is never dropped by acknowledgment rules;
- state-version increase prevents stale-update suppression;
- incident occurrence count is monotonic;
- private review events cannot cross a public boundary without synthesis.

A.16.3 Replay tests

Replay synthetic transcript-derived sequences under legacy and governor modes. Assert outbound message counts and owners.

A.16.4 Canary tests

Use one low-risk private channel before mixed human channels. Preserve full logs and provide immediate rollback.

A.17 Suggested commit sequence

A clean incremental sequence might be:

1. Documentation and event-flow map.
2. Event schema and ledger, no behavior change.
3. Replay harness and synthetic fixtures.
4. Shadow admission decisions.
5. Deterministic egress no-op suppression.
6. Response leases.
7. Private review bus.
8. Project-state snapshot pilot.
9. Daily digest pilot.
10. Incident manager.
11. Semantic dedup shadow mode.

12. Metrics and rollout report.

Each commit should include tests and a concise migration note.

A.18 Additional implementation gates

The implementing agent must stop for explicit review at these points:

1. after topology discovery, before choosing centralized or coordinated-local integration;
2. after the replay corpus and shadow metrics are available, before active suppression;
3. before enabling semantic deduplication or any model-based governor classifier;
4. before enabling a new human-visible channel or public social-media boundary;
5. after any proposed egress transform that cannot be mechanically shown to preserve substantive meaning.

A.19 Definition of done for the first production release

The first production release is complete when:

1. The selected physical topology is documented and supported by traced events from both Protocos-mobot and OmegaClaw.
2. Both agents participate in one logically shared policy, ledger, and public-egress regime, even if local governors are physically distributed.
3. Exact structured no-send and verified no-op messages do not reach Telegram.
4. Consequential acknowledgments and protocol state changes remain visible or durably recorded as policy requires.
5. A human message receives one public answer by default, with task/thread hysteresis preventing rapid voice thrashing.
6. Explicit requests for multiple agent views and scoped human override commands work.
7. Repeated provider errors are coalesced.
8. At least one project worker reports through a daily digest instead of direct routine updates.
9. A private review request can influence one public synthesis.
10. The semantic firewall prevents substantive egress rewriting; substantive contract failures return to the source agent for regeneration.
11. Governor CPU, latency, model, embedding, and token overhead are measured and remain within the approved budget.
12. Metrics show meaningful net reduction in calls and messages without observed loss of important information.
13. All active behavior is explicitly approved, feature-flagged, auditable, and reversible.

14. Documentation describes topology, configuration, human overrides, troubleshooting, calibration, and rollback.

A.20 Implementation-agent reporting format

The OpenClaw implementation agent should report milestones using:

```
Completed:
Evidence:
Behavioral change:
Tests:
Risks or blocker:
Next gate:
```

It should not send a message for every command, file inspection, or minor edit.

B Example Policy Configuration

```
{
  "version": "ocg-policy-0.1",
  "global": {
    "default_public_slots": 1,
    "suppress_visible_no_reply": true,
    "suppress_ack_only": true,
    "tool_narration_public": false,
    "semantic_dedup_mode": "shadow",
    "topology_mode": "discovery_required",
    "explicit_human_addressing_bypass": true,
    "governor_cost_budget": {
      "max_model_calls_per_100_events": 2,
      "minimum_efficiency_ratio": 5.0
    },
    "egress_semantic_firewall": "strict",
    "human_overrides": true
  },
  "agents": {
    "protocosmo": {
      "roles": ["execution", "evidence", "operations", "project_owner"],
      "default_message_classes": ["status_delta", "decision_request"]
    },
    "omegaclaw": {
      "roles": ["synthesis", "critique", "theory", "long_form"],
      "default_message_classes": ["substantive_answer", "private_review"]
    }
  },
  "channels": {
    "telegram:protobots": {
      "type": "mixed_human_ai",
      "default_spokesperson": "protocosmo",
      "routine_updates": "daily_digest",
      "max_public_responders": 1,
      "max_bot_turns_per_state_version": 2,
      "owner_stickiness_minutes": 10,
      "override_default_scope": "one_message"
    }
  }
}
```

```

    },
    "internal:reviews": {
      "type": "ai_only_structured",
      "human_visible": false,
      "max_turns": 3,
      "retain_full_log": true
    }
  }
}

```

C Test Matrix

Scenario	Expected governor behavior	Critical assertion
Human addresses Protocosmobot	Admit Protocosmobot; optional private OmegaClaw review	Human receives one public answer.
Human addresses OmegaClaw	Admit OmegaClaw	Protocosmobot does not paraphrase the answer.
Human asks both agents separately	Issue two leases or comparative-synthesis policy	Explicit diversity request is preserved.
Bot posts NO_REPLY	Suppress at egress	Zero Telegram sends.
Sibling posts “noted”	Drop before inference when detectable	No LLM call.
Same 503 repeats ten times	Update one incident	At most one open notice before recovery/escalation.
Tool emits five progress steps	Log internally	Only completion or blocker becomes public.
Project commit count increases twice	Aggregate state	One daily delta.
Project has no material change	Log only	No public update.
Owner crashes after lease	Expire and reassign	At most one eventual public answer.
Message arrives in three parts	Buffer until complete	No response to partial content.
Near-duplicate answer with new number	Do not suppress	New result reaches human.
Near-duplicate answer with no new content	Suppress or shadow-flag	Duplicate count decreases.
Non-owner detects security issue	Override lease policy	High-severity warning reaches owner/human.
AI-only review contains private speculation	Keep internal	Public synthesis does not leak private content.
Ownership-acceptance acknowledgment	Record structured state and optionally confirm	Consequential acknowledgment is not falsely suppressed.
Rapid follow-ups in one thread	Retain owner under hysteresis	Public voice does not thrash.

Scenario	Expected governor behavior	Critical assertion
Human sends /solo @omegaclaw	Override lease resolution	Named agent owns the scoped reply.
Human sends /all-answer	Permit scoped multi-response	Exception applies only to the requested message by default.
Candidate length but contains safety caveat	Regenerate at source	Governor does not truncate away the caveat.
Governor classifier costs more than saved call	Use cheaper or deterministic path	Net-savings gate blocks activation.
No shared pre-inference boundary exists	Select Topology B or C	Design remains implementable without false centralization assumption.

D Future Research Directions

Once the practical governor is stable, the event log enables deeper study of multi-agent communication:

- learning policies that optimize useful information per token rather than brevity alone;
- estimating marginal value of an additional reviewer;
- detecting conversational attractors such as acknowledgment loops and status spirals;
- attention allocation as a market or ECAN-like process;
- causal attribution of which agent contribution improved the final answer;
- adaptive committee formation based on task novelty and risk;
- formal safety properties for one-response leases and privacy boundaries;
- modeling the governor as an OmegaHive social-regulation layer rather than a mere cost-control mechanism;
- formalizing acknowledgment loops as zero-information conversational attractors;
- interpreting leases as mutual-exclusion invariants over shared conversational attention;
- connecting novelty and expected information gain to ECAN-style attention allocation and Hyperseed representations.

The most important research stance is that low-noise communication should be evaluated as a form of functional coordination, not cosmetic concision. The desired system speaks less because it routes attention and responsibility better, not because its agents have less to contribute.